

Programming Fundamentals

algorithms, flowcharts and pseudocode

Programming Fundamentals

turning code into something a machine can run

Programs and Code

We know a computer follows instructions. So how do we actually hand it a set of them? We write code.

Program

A set of instructions written to make a computer perform a task.

Code

The instructions we write using a programming language.

A VERY SMALL PROGRAM

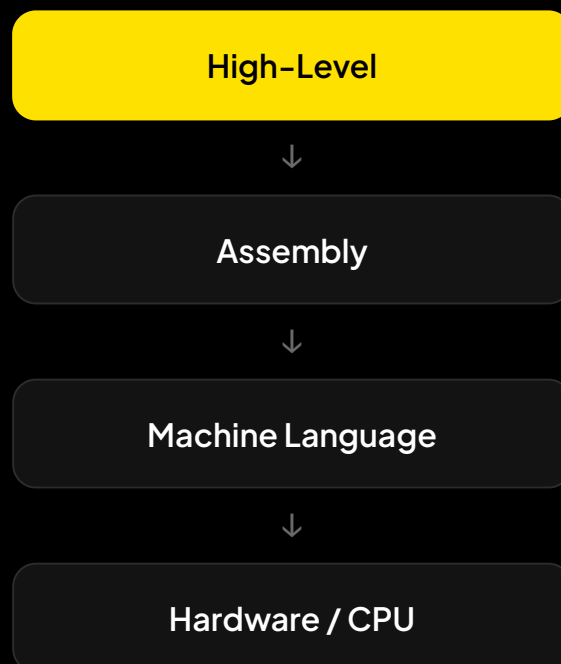
```
int a = 10;  
int b = 20;  
int sum = a + b;  
cout << sum;
```

Four lines. Store a value, store another, add them together, show the result. That is a complete program — data held in memory, processed, and output produced.

From Human Language to Hardware

The code above is readable by a human. The CPU cannot read it at all — the CPU only understands binary. So there are levels in between.

LEVEL	WHAT IT IS
High-Level Language	Easier for humans to read and write; translated into machine instructions before execution. C++, Python, Java.
Assembly Language	Human-readable mnemonics representing machine instructions.
Machine Language	Binary instructions the CPU directly executes.



The further down you go, the closer to the hardware and the harder it is for a human to read. C++ sits at the top and gets translated downward.

Compiler

DEFINITION

A compiler **translates source code into machine code**, or another lower-level form, **before** the program runs.



One full translation, up front — then the CPU runs the result directly. This is what C++ uses.

Interpreter

DEFINITION

An interpreter **executes a program by translating and running its instructions during runtime**, rather than producing a complete executable beforehand.



No separate executable is produced — each line is translated and run as it goes. This is what Python uses.

Compiler vs Interpreter

COMPILER	INTERPRETER
Translates before execution	Translates and executes during runtime
Produces an executable or lower-level output	Usually runs through the interpreter itself
Example: C++	Example: Python



If you are asked "which one translates the entire program before it runs?" the answer is the **compiler**. That single word, **before**, is the whole difference.

How Code Runs



Your code is compiled, loaded into RAM, and then run to produce the output.

Notice RAM appearing here — that is the primary memory from Chapter 2 doing its actual job.

SUMMARY

- A program is a set of instructions; code is how we write them.
- High-Level → Assembly → Machine Language → Hardware.
- Compiler translates the whole program before running (C++).
- Interpreter translates and runs line by line (Python).
- Code → Compiler → RAM → Output.

Algorithms

step-by-step thinking, before a single line of code

DEFINITION

An algorithm is a **finite, step-by-step procedure** used to solve a problem or perform a task.

The word "finite" matters — an algorithm has to end. A set of steps that runs forever is not an algorithm.

Steps to Solve an Algorithmic Problem

Before you touch a compiler, this is the order to work in.

- 1 Analyse the problem statement.** Make the objective clear in your mind — what exactly is the input, and what is the required output?
- 2 Break it down.** Some problems are complex. Split them into smaller sub-parts so each piece is easy to understand.

- 3 **Write the solution on paper first.** Note down the basic steps. This gives you a clear intuition of what you are going to do, and saves a lot of time debugging later.
- 4 **Verify with test cases.** Check your solution against a bunch of examples to make it error-free before you write any code.
- 5 **Now write the code.** By this point you know exactly what you are doing, so coding becomes the easy part.



Most beginners jump straight to step 5. That is why they spend hours debugging. The paper stage feels slow, but it is the fastest route to working code.

A Real-Life Algorithm

You already write algorithms every day without calling them that. Making tea is one.

Problem

Make tea.

Algorithm

1. Boil water
2. Add tea
3. Add milk
4. Add sugar
5. Serve



Order matters. Serve first and boil later and you get a very different result. That is why an algorithm is a **sequence**, not just a list of things to do.

SUMMARY

- An algorithm is a finite, step-by-step procedure to solve a problem.
- Analyse → break down → write on paper → verify with test cases → code.
- Order matters; an algorithm is a sequence.

Flowcharts

drawing the logic out, so it is easy to follow before it is coded

DEFINITION

A flowchart is a **diagrammatic representation of an algorithm** — a step-by-step approach to solving a problem, drawn out visually.

Uses of Flowcharts

- Used in **documentation**.
- Used to **communicate your solution** with others — especially useful for group projects.
- Used to **check at any step** what is going to happen, giving a clear explanation of the flow of statements.
- Makes logic easier to visualise, and shows decisions clearly before you start coding.

Flowchart Components

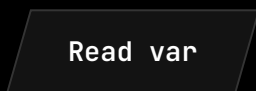
Each shape has a fixed meaning. Using the wrong shape is like using the wrong punctuation — people will still read it, but it signals the wrong thing.



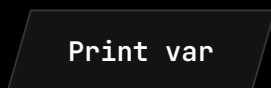
Terminator. Mainly used to denote the start point of the program.



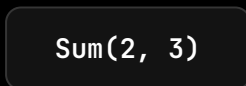
Terminator. Used to denote the end point of the program.



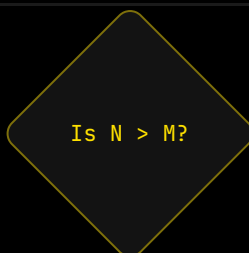
Input. Used for taking input from the user and storing it in a variable named `var`.



Output. Used to output the value stored in the variable `var`.



Process. Used to perform the operations in the program. For example `Sum(2, 3)` performs the arithmetic summation of 2 and 3.



Decision. Used to make decisions. It depends on some condition and answers in the form of TRUE (for yes) and FALSE (for no).



Arrow. Shows the flow of the program from one step to another. The head shows the next step and the tail shows the previous one.



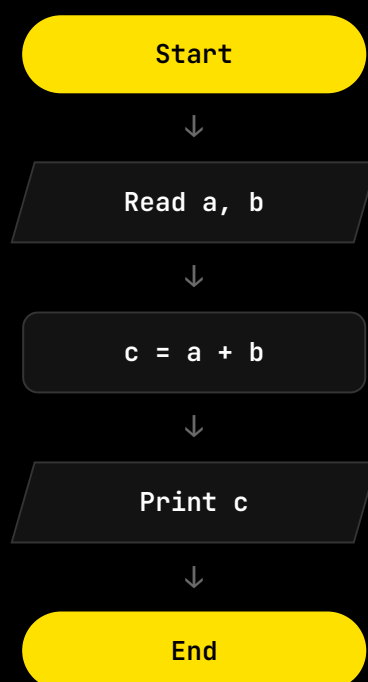
Connector. Used to connect different parts of the program in case of a break-through. Generally used for functions, which come later in the course.



The diamond is the shape that carries real information — it is the only one with **two exits**. Every branch in your logic is a diamond.

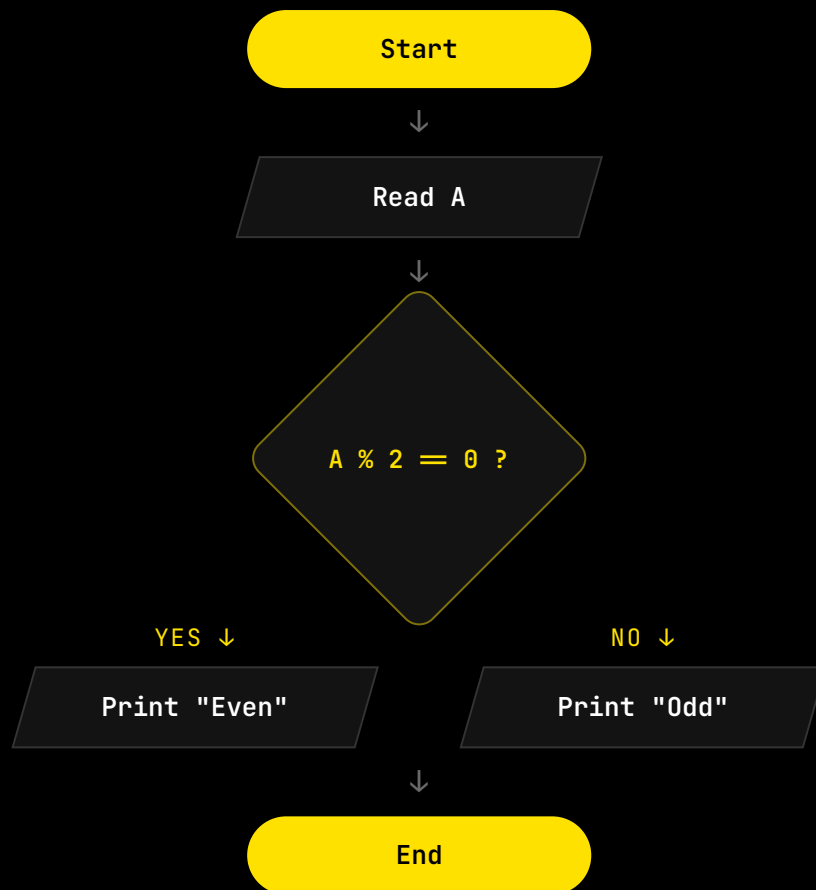
Example 1 — Adding Two Numbers

Suppose we have to make a flowchart for adding two numbers **a** and **b**. There are no decisions here, so it is a straight line from start to end.



Example 2 — Even or Odd

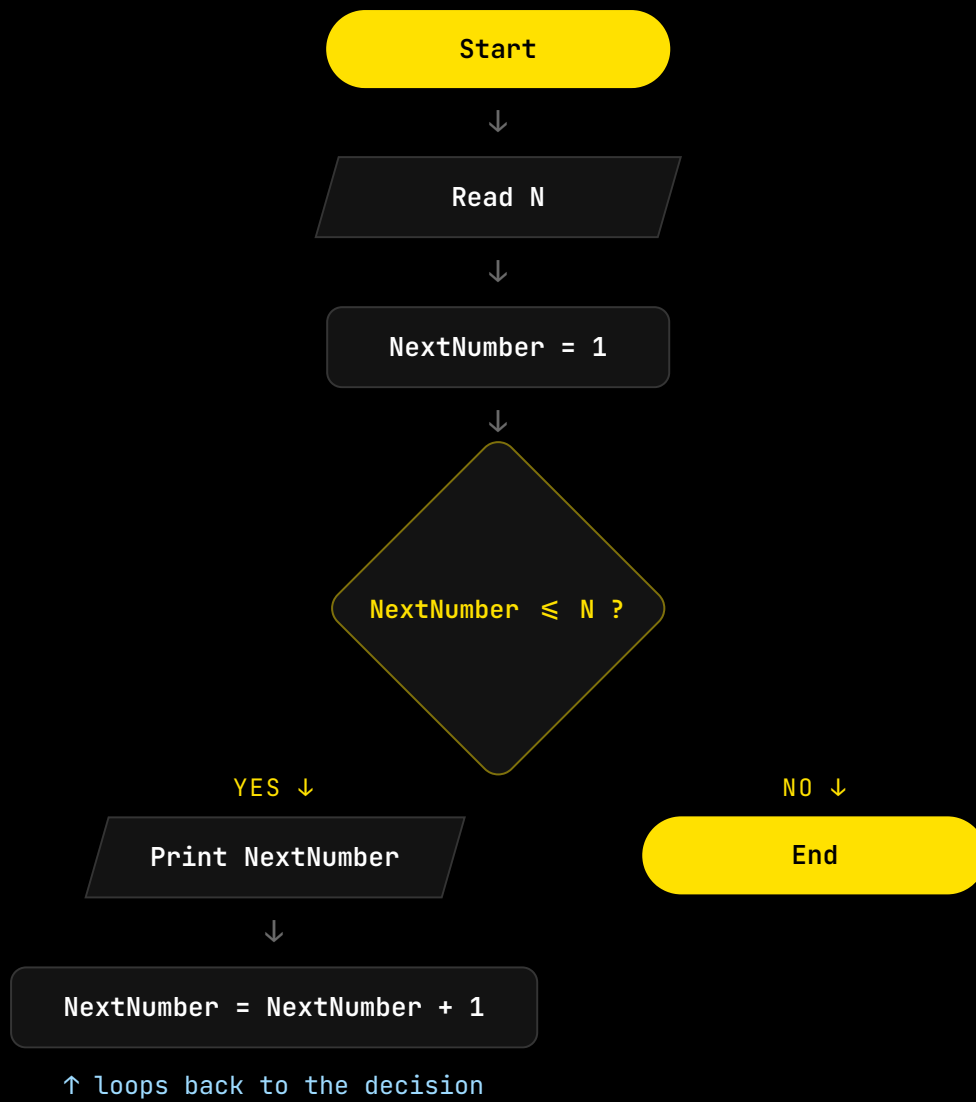
Suppose we have to check if a number is even or odd. Now we need a decision, so a diamond appears and the path splits into two.



Note on %: the operator `%` finds the remainder of one number divided by another. For example: $4 \% 3 = 1$, $10 \% 5 = 0$, $4 \% 2 = 0$, $4 \% 4 = 0$, $0 \% 1 = 0$. And $1 \% 0$ is undefined, because it leads to division by zero.

Example 3 — Print Numbers Up To N

Print all numbers less than or equal to `N`, where `N` is given by the user. This one is different — the arrow loops **backward**, so the same steps repeat until the condition finally fails.



That backward arrow is a **loop**. Without it the flowchart would need one box per number, which is impossible when N is decided by the user at runtime. This is the exact idea behind **for** and **while** loops in C++.

SUMMARY

- Flowcharts are the building block of any program written in any language.
- Different shapes have different fixed meanings.
- Every problem can be represented in the form of a flowchart.
- Sometimes it becomes a bulky process to represent a program using a flowchart. In those cases, look for the optimal solution instead.

Pseudocode

writing the logic in plain structured text

Flowcharts show logic visually. Pseudocode writes the **same** logic using simple, structured text — without the exact syntax of any programming language.

IMPORTANT

Pseudocode is **not C++**. It will not compile and it is not meant to. It focuses purely on logic, so you can think about the problem without fighting syntax.

Example 1 — Sum of Two Numbers

SUM OF A AND B

START

INPUT A

INPUT B

SUM = A + B

PRINT SUM

END

Compare this to Example 1 in the previous chapter — identical steps, just typed instead of drawn.

Example 2 — Maximum of Two Numbers

LARGER OF A AND B

START

INPUT A

INPUT B

IF A > B

PRINT A

ELSE

PRINT B

END

The diamond from the flowchart becomes **IF ... ELSE**. That is the direct translation — every decision shape turns into an IF.

Example 3 — Even or Odd

EVEN OR ODD

```
START
INPUT N
IF N % 2 = 0
    PRINT "Even"
ELSE
    PRINT "Odd"
END
```

Example 4 — Positive, Negative or Zero

Three outcomes now, so a single IF is not enough. This is where **ELSE IF** comes in — the checks happen top to bottom, and the first one that is true wins.

SIGN OF N

```
START
INPUT N
IF N > 0
    PRINT "Positive"
ELSE IF N < 0
    PRINT "Negative"
ELSE
    PRINT "Zero"
END
```

CONDITION	RESULT
$N > 0$	Positive

CONDITION	RESULT
$N < 0$	Negative
else	Zero

Example 5 — Prime Number

This is the beginner version. The $\sqrt{N}$ optimisation comes later in the course.

PRIME CHECK

START

INPUT N

IF N < 2

PRINT "Not Prime"

ELSE

CHECK WHETHER N HAS A DIVISOR

IF DIVISOR EXISTS

PRINT "Not Prime"

ELSE

PRINT "Prime"

END



Notice the IF sitting **inside** another ELSE in this example. That is called **nesting**, and it is exactly the same shape you will write in C++ — just with braces around it.

SUMMARY

- Pseudocode writes the same logic as a flowchart, in plain structured text.
- It is not C++ and is not meant to compile — it focuses only on logic.
- Every diamond in a flowchart becomes an IF in pseudocode.
- Use ELSE IF when there are more than two possible outcomes.